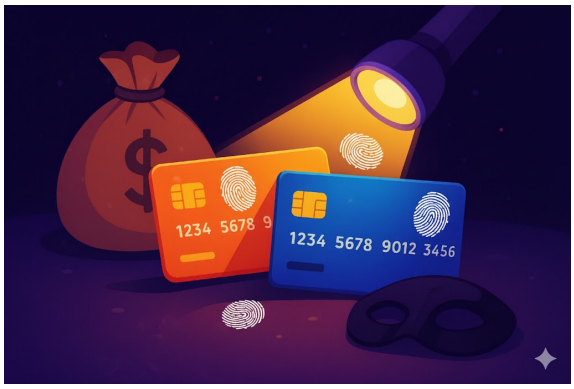
Metodología de la Programación

Curso 2025/2026



Guion de prácticas

Fraud4

Sobrecarga de operadores. Clasificación con algoritmo KNN

Mayo de 2026

Silvia Acid, Andrés Cano, Luis Castillo
Dpto. Ciencias de la Computación e Inteligencia Artificial
Universidad de Granada

Licencia Creative Commons con Reconocimiento - No Comercial - Compartir Igual 4.0 (CC BY-NC-SA 4.0)

Índice

1. Introducción **5**

2. Arquitectura de las prácticas **5**

3. Objetivos **6**

4. Clasificación de instancias de un dataset **7**
4.1. Algoritmo KNN para clasificación de un dataset 8
4.2. Obtención de la k-ésima instancia más cercana 10
4.3. Un ejemplo detallado de clasificación con KNN 10

5. Práctica a entregar **13**
5.1. Finalidad del programa 13
5.2. Sintaxis y ejemplos de ejecución 13
5.3. Módulos del proyecto 16
5.4. Entrega de la práctica 17

6. Código para la práctica **18**
6.1. Location.h 18
6.2. VectorLocation.h 21
6.3. VectorInt.h 25
6.4. Clustering.h 28
6.5. DataSet.h 33
6.6. main.cpp 38

1. Introducción

El propósito de esta práctica final es el de realizar una aplicación que sea capaz de clasificar las instancias de un dataset de test, usando otro dataset como modelo. El algoritmo de clasificación que usaremos es el algoritmo KNN (K-vecinos más cercanos), un algoritmo relativamente sencillo. La clasificación de una instancia consiste en asignarle la etiqueta más probable de forma automática.

Para desarrollar este algoritmo usaremos gran parte del trabajo que hemos desarrollado en prácticas anteriores. Por ejemplo, el algoritmo de clasificación puede utilizar el algoritmo de clustering y el de reducción de dimensionalidad que implementamos anteriormente.

En esta práctica conservamos todas las clases desarrolladas en la práctica anterior, aunque añadiremos nueva funcionalidad en todas ellas. En la clase `DataSet` se incluirá una función para clasificar un dataset y en todas las clases incluiremos la sobrecarga de algunos operadores.

Por último, el programa de esta práctica necesita controlar un mayor número de parámetros por línea de comandos que el de la práctica anterior, como veremos en la sección 5.2.

2. Arquitectura de las prácticas

En `Fraud4` no aparece ninguna clase nueva. En todas las clases de la práctica anterior tenemos que realizar la sobrecarga de algunos operadores. Además, en la clase `DataSet` tenemos que añadir nueva funcionalidad para poder clasificar un dataset. Como siempre, puede ver en la figura 1 la lista de módulos que usaremos.

Describamos brevemente los módulos que sufrirán modificaciones en esta práctica:

A Location
: La clase Location solo requiere que se sobrecarguen los operadores de entrada y salida, así como los operadores `+=` y `/=`.

B' VectorLocation
: La clase VectorLocation requiere que se sobrecarguen los operadores de entrada y salida, así como los operadores `[]` y `+=`.

C' VectorInt
: La clase VectorInt requiere que se sobrecarguen los operadores de entrada y salida, así como los operadores `[]`, `+=`.

D Clustering
: La clase Clustering requiere que se sobrecarguen todos los operadores relacionales.

E DataSet
: La clase DataSet requiere que se sobrecarguen los operadores de entrada salida y el operador `()`. Además, debe añadir la función

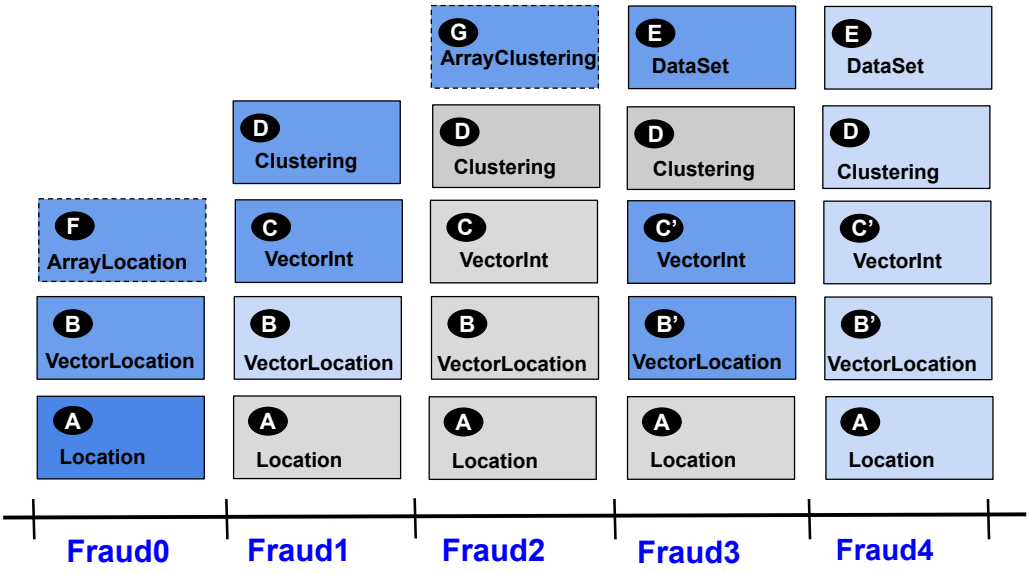


Figura 1: Arquitectura de las prácticas de MP 2026. Como podemos observar, los bloques correspondientes a cada módulo, aparecen en distintos colores. Los cambios pequeños, como la incorporación de nuevas funcionalidades, se muestran en azúl suave.

para clasificar un dataset, y el método para obtener la instancia más cercana a otra dada.

main.cpp Contiene la función main() cuya finalidad será clasificar un dataset de test.

3. Objetivos

El desarrollo de la práctica `Fraud4` persigue los siguientes objetivos:

- Practicar con la sobrecarga de distintos operadores tanto monarios como binarios.
- Profundizar en el uso del paso de parámetros a `main()` desde la línea de comandos (parámetros alternativos, optativos, de distintas longitudes y por defecto).
- Practicar con herramientas como `valgrind` y el depurador de VS-Code para rastrear errores en tiempo de ejecución.
- Entrenarse con el desarrollo de algoritmos: implementación del algoritmo KNN de clasificación.

4. Clasificación de instancias de un dataset

En la práctica `Fraud3` indicamos que un dataset está compuesto de un conjunto de instancias. Cada instancia tiene asociada una etiqueta, que informa del valor de la clase de tal instancia. En nuestro problema, estamos usando números enteros como posibles valores para las etiquetas de las instancias de un dataset. Concretamente, en los datasets que estamos usando en las prácticas, solo usamos los valores 0 y 1. Una instancia con una etiqueta con valor 0 representa un conjunto de compras *no-fraudulentas*. Una instancia con una etiqueta con valor 1 representa un conjunto de compras *fraudulentas*.

El programa de esta práctica tiene por finalidad la clasificación de todas las instancias de un dataset. A tal dataset le llamaremos *dataset de test* o bien *dataset a clasificar*. La clasificación de una instancia consiste en asignarle la etiqueta más probable. Para ello, usaremos el conocido algoritmo de clasificación KNN (*K-nearest neighbours*). Este es un algoritmo relativamente sencillo, como veremos a continuación.

El algoritmo KNN usa otro dataset como modelo para clasificar, que llamaremos *dataset modelo* o bien *dataset de entrenamiento*. Este dataset debe tener correctamente clasificadas todas sus instancias para que pueda servir como un buen modelo de clasificación. O sea, cada instancia del dataset modelo debe tener asignado el valor correcto en su etiqueta. De esta forma, el algoritmo KNN toma cada instancia del dataset a clasificar, y busca las K[1] instancias más cercanas del dataset modelo. Entonces clasifica tal instancia con la etiqueta mayoritaria entre las más cercanas. Este proceso se repite para cada instancia del dataset a clasificar.

Ejemplo 1 *Ejemplo del algoritmo KNN*

La figura 2 muestra gráficamente un ejemplo de aplicación del algoritmo KNN para clasificar la instancia que representa el círculo verde. Si usamos K = 3, la instancia sería clasificada con la clase triángulo, ya que hay solo 1 cuadrado y 2 triángulos dentro del círculo que los contiene. Si usamos K = 5, la instancia sería clasificada con la clase cuadrado, ya que hay 2 triángulos y 3 cuadrados dentro del círculo externo.

Nota: *Imagen y texto parcialmente tomado del artículo de Wikipedia "K vecinos más próximos", consultado el 25 de marzo de 2026 en https://es.wikipedia.org/wiki/K_vecinos_más_próximos*

Como hemos indicado anteriormente, el algoritmo KNN necesita buscar las K instancias más cercanas a otra instancia dada. Para ello es necesario definir una medida de distancia entre dos instancias. Nosotros usaremos la distancia euclídea entre instancias. Recordamos que en la clase `VectorInt` hemos implementado en prácticas anteriores el método `distance()`, que nos permite obtener la distancia euclídea entre dos instancias.

Hay que hacer notar que para poder aplicar el algoritmo KNN, es obligatorio que el dataset de entrenamiento y el dataset de test tengan el

[1] No confundir este valor de K, con el K que hemos usado para realizar clustering en prácticas anteriores

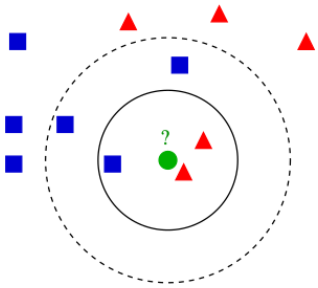


Figura 2: Ejemplo del algoritmo KNN

mismo número de localizaciones. Incluso deberían ser las mismas localizaciones y aparecer en el mismo orden en los dos datasets, aunque esto no será necesario comprobarlo en nuestro código. La razón de que ambos datasets deban tener el mismo número de localizaciones, es para que pueda calcularse la distancia entre una instancia de uno y otro; necesitamos que las dos instancias a comparar tengan la misma dimensión.

4.1. Algoritmo KNN para clasificación de un dataset

El algoritmo KNN para clasificar todas las instancias de un dataset se implementará en el módulo `DataSet`, en la siguiente función externa:

```
/**
 * Classifies the instances of the dataset @p dataSetToClassify with
 * the KNN algorithm and using the dataset @p dataSetModel as model to classify.
 * Each instance in @p dataSetToClassify will be classified by changing its
 * label to the one found by the KNN algorithm.
 * @throw std::invalid_argument Throws a std::invalid_argument exception if the
 * number of locations in datasetModel and datasetToClassify is not equal.
 * @throw std::invalid_argument Throws a std::invalid_argument exception
 * if @p doReductionDimensionality is true and @p K1 is less or equals than 0.
 * @throw std::invalid_argument Throws a std::invalid_argument exception
 * if @p K2 is greater than the number of instances in @p datasetModel
 * @throw std::invalid_argument Throws a std::invalid_argument exception
 * if @p K2 is less than 1
 * @param dataSetModel DataSet used as model to classify. Input parameter
 * @param dataSetToClassify DataSet to be classified. Input/output parameter
 * @param K1 The K value to be used in the clustering algorithm (KMeans) of
 * the dimensionality reduction step. Input parameter
 * @param K2 The K value to be used in the KNN algorithm (to define the number K
 * of nearest neighbours). Input parameter
 * @param doReductionDimensionality Indicates whether or not dimensionality
 * reduction should be applied using the clustering algorithm. Input parameter
 */
void classify(DataSet datasetModel, DataSet datasetToClassify, int K1, int K2, bool
     doReductionDimensionality);
```

En la implementación del algoritmo de clasificación en esta función, el parámetro `doReductionDimensionality` permite elegir si se desea aplicar previamente un proceso de reducción de la dimensionalidad a los dos datasets (modelo y a clasificar). La reducción de dimensionalidad podría mejorar el porcentaje de instancias bien clasificadas. En caso de elegir que sí, la función debe obtener dos datasets de dimensionalidad reducida (usando dos objetos locales de la función) a partir de los datasets modelo y a clasificar. Entonces, estos dos nuevos datasets serían los que se usarían como datasets modelo y a clasificar en el proceso de clasificación. Una vez acabe el proceso de clasificación de instancias del dataset a clasificar con dimensionalidad reducida, se copiarán los valores de las etiquetas del dataset clasificado en el dataset a clasificar original.

Describimos a continuación los parámetros de la función `classify()`:

- `datasetModel`: dataset que usaremos como modelo (dataset de

entrenamiento).

- `dataSetToClassify`: dataset a clasificar. En este dataset es en el que debemos asignar una nueva etiqueta a cada una de sus instancias.
- `K1`: número entero con el valor K que usaremos en el proceso de clustering usado para configurar la reducción de dimensionalidad de los dos datasets (`datasetModel` y `datasetToClassify`), si es que esta es aplicada.
- `K2`: número entero con el valor K para el algoritmo KNN.
- `doReductionDimensionality`: valor booleano que nos permite decidir si queremos o no aplicar un proceso de reducción de dimensionalidad a los datasets modelo y a clasificar, previamente al proceso de clasificación de instancias.

La función `classify()` puede implementarse mediante los pasos descritos en el algoritmo 1.

Algorithm 1 Algoritmo de clasificación de un dataset

Input: datasetModel (dataset usado como modelo)
Input: datasetToClassify (dataset a clasificar)
Input: K1 (parámetro K del clustering)
Input: K2 (parámetro K del algoritmo KNN)
Input: doReductionDimensionality (aplicar o no reducción de dimensionalidad)
Output: dataset datasetToClassify clasificado
$dsM \leftarrow$ datasetModel ▷ Hacer una copia de datasetModel
$dsC \leftarrow$ datasetToClassify ▷ Hacer una copia de datasetToClassify
if doReductionDimensionality==true **then**
    $dsM \leftarrow dsM_R$ ▷ Reducir dimensionalidad de dsM ($K = K1$)
    $dsC \leftarrow dsC_R$ ▷ Reducir dimensionalidad de dsC ($K = K1$)
end if
for $fila = 0$ hasta $NFilas(dsC) - 1$ **do** ▷ Para cada instancia de dsC
    v[0] $\leftarrow$ 0
    v[1] $\leftarrow$ 0
    for $k = 1$ hasta K **do** ▷ Obtención de las K instancias más cercanas
        i $\leftarrow$ índice en dsM de la k-ésima instancia más cercana a $dsC[fila]$
        l $\leftarrow$ Label($dsM[i]$)
        v[l] $\leftarrow$ v[l] + 1
    end for
    if v[0] $\geq$ v[1] **then** ▷ Asignar etiqueta a la instancia nº fila de dsC
        Label(dsC[fila]) $\leftarrow$ 0
    else
        Label(dsC[fila]) $\leftarrow$ 1
    end if
end for
for $fila = 0$ hasta $NFilas(dsC) - 1$ **do** ▷ Asignar etiquetas a instancias de datasetToClassify
    Label(datasetToClassify[fila]) $\leftarrow$ Label(dsC[fila])
end for

Como hemos indicado anteriormente, el algoritmo KNN debe buscar para cada instancia del dataset a clasificar, las K instancias más cercanas del dataset modelo, y asignarle entonces como etiqueta, el valor de la

etiqueta mayoritaria entre las de las K más cercanas. En el algoritmo KNN (algoritmo 1), el bucle **for** de las líneas 16 a 20 es el encargado de buscar en el dataset modelo, las K instancias más cercanas a la instancia $dsC[fila]$. Además, ayudándose del vector `v`, calcula el número de tales instancias con etiqueta 0 y con etiqueta 1.

Fíjese, que cuando el parámetro de entrada `doReductionDimensionality` es `false`, no se aplica el proceso de reducción de dimensionalidad de los datasets modelo y a clasificar, y por tanto el parámetro `K1` no sería usado en el algoritmo.

4.2. Obtención de la k-ésima instancia más cercana

En la línea 17 del algoritmo 1 debemos buscar en el dataset modelo, el índice de la k-ésima instancia más cercana a la instancia $dsC[fila]$. Para ello usaremos el método `nearestInstance()`, que debe implementar en la clase `DataSet`.

```
/**
 * Returns the index (position) of the nearest instance in this DataSet to
 * the provided instance (@p instance). To do so, the Euclidean distance
 * is used to calculate the distance between two instances.
 * If an instance i has already been selected (selected[i] == true) then
 * it is not considered as a possible nearest instance.
 * @throw std::invalid_argument Throws a std::invalid_argument exception
 * if the size of the provided instance and the number of locations in this
 * Dataset is not equal.
 * Query method
 * @param instance A VectorInt that represents the instance for which
 * we want to calculate the nearest instance.
 * Input parameter
 * @param selected array of bool that defines the already selected instances.
 * This array should have at least a capacity equal to the number of
 * instances in this DataSet.
 * Input parameter
 * @return An integer with the index (position) of the nearest instance to
 * @p instance. It returns -1 if this DataSet does not contain any instance.
 */
int nearestInstance(VectorInt instance, bool selected[]);
```

Como puede ver, este método recibe como parametro un array de booleanos (`selected`) que permite indicar qué instancias han sido ya seleccionadas previamente para que así, en las siguientes iteraciones del algoritmo KNN (del bucle **for** de las líneas 16 a 20), no devuelva siempre el mismo valor. El array `selected` debe tener un tamaño igual al número de instancias del dataset modelo.

El algoritmo 1 debería crear el array local de booleanos `selected` y usarlo para llamar a esta función. Tal array debería estar inicializado con todos los valores a `false` antes de entrar en el bucle **for** de la línea 16. En cada iteración de tal bucle **for**, tras llamar al método `nearestInstance()`, el algoritmo debería poner a `true` la posición correspondiente a la instancia más cercana encontrada, para que así quede descartada en las siguientes iteraciones.

4.3. Un ejemplo detallado de clasificación con KNN

Vamos a detallar el proceso de clasificación mediante un ejemplo. Supongamos que disponemos del dataset del ejemplo 2 en el que asumimos que sus instancias están correctamente clasificadas. Este dataset lo usaremos como dataset de entrenamiento para la clasificación.

Ejemplo 2 *Dataset de entrenamiento con 5 localizaciones y 10 instancias de compras*

Este dataset puede encontrarse en el fichero `Datasets/dataP4/simple_5Locations_tr10.dst`.

```
MP-FRAUD_DATASET-T-1.0
5
0.0 0.0 locA
1.0 1.0 locB
2.0 2.0 locC
3.0 3.0 locD
4.0 4.0 locE
10
0 1 1 1 0 0 0 0 0 0
1 2 3 4 5
1460 16 0 1615 1155
23 1285 30 1318 0
1163 26 1661 83 1200
20 0 20 0 20
0 0 11 11 0
28 700 560 0 13
21 21 71 0 32
10 10 10 0 0
1 1 1 1 1
```

Supongamos que queremos clasificar las instancias del dataset de test del ejemplo 3, que cuenta con 5 localizaciones y 2 instancias.

Ejemplo 3 *Dataset de test con 5 localizaciones y 2 instancias.*

Este dataset tiene las mismas localizaciones que el del ejemplo 2. Puede encontrarse en el fichero `Datasets/dataP4/simple_5Locations_tst2.dst`.

```
MP-FRAUD_DATASET-T-1.0
5
0.0 0.0 locA
1.0 1.0 locB
2.0 2.0 locC
3.0 3.0 locD
4.0 4.0 locE
2
0 1
10 10 5 1 1
1000 20 1000 80 1000
```

Como podemos ver, el dataset `simple_5Locations_tst2.dst`, como todo fichero dataset, contiene ya una línea con las etiquetas de cada una de las instancias. Estas etiquetas han sido proporcionadas por algún experto, y son las etiquetas correctas. Una vez realizado el proceso de clasificación de este dataset de test, podríamos comprobar si el algoritmo de clasificación automática utilizado, acierta en sus predicciones.

En la tabla 1 se muestran las distancias entre cada instancia del dataset de test y cada instancia del dataset de entrenamiento. La columna **nº** indica el número de instancia del dataset. La columna **dist.** indica la distancia entre una instancia de dsC y una instancia de dsM. La columna **k** indica el valor k del k-ésimo vecino más cercano. Así, si **k** es 1, la instancia de dsM es la primera más cercana; si **k** es 2 sería la segunda más cercana, y así sucesivamente. La columna **l** indica la etiqueta que tiene la correspondiente instancia de dsM.

Dataset dsC		Dataset dsM				
nº	**instancia**	**nº**	**instancia**	**dist.**	**k**	**l**
0	$(10, 10, 5, 1, 1)$	0	$(1, 2, 3, 4, 5)$	13.19	2	0
0	$(10, 10, 5, 1, 1)$	1	$(1460, 16, 0, 1615, 1155)$	2457.49	10	1
0	$(10, 10, 5, 1, 1)$	2	$(23, 1285, 30, 1318, 0)$	1833.28	8	1
0	$(10, 10, 5, 1, 1)$	3	$(1163, 26, 1661, 83, 1200)$	2348.69	9	1
0	$(10, 10, 5, 1, 1)$	4	$(20, 0, 20, 0, 20)$	28.05	5	0
0	$(10, 10, 5, 1, 1)$	5	$(0, 0, 11, 11, 0)$	18.36	4	0
0	$(10, 10, 5, 1, 1)$	6	$((28, 700, 560, 0, 13)$	885.77	7	0
0	$(10, 10, 5, 1, 1)$	7	$(21, 21, 71, 0, 32)$	74.56	6	0
0	$(10, 10, 5, 1, 1)$	8	$(10, 10, 10, 0, 0)$	5.20	1	0
0	$(10, 10, 5, 1, 1)$	9	$(1, 1, 1, 1, 1)$	13.34	3	0
1	$(1000, 20, 1000, 80, 1000)$	0	$(1, 2, 3, 4, 5)$	1728.62	7	0
1	$(1000, 20, 1000, 80, 1000)$	1	$(1460, 16, 0, 1615, 1155)$	1895.22	9	1
1	$(1000, 20, 1000, 80, 1000)$	2	$(23, 1285, 30, 1318, 0)$	2455.26	10	1
1	$(1000, 20, 1000, 80, 1000)$	3	$(1163, 26, 1661, 83, 1200)$	709.60	1	1
1	$(1000, 20, 1000, 80, 1000)$	4	$(20, 0, 20, 0, 20)$	1699.41	4	0
1	$(1000, 20, 1000, 80, 1000)$	5	$(0, 0, 11, 11, 0)$	1727.22	6	0
1	$(1000, 20, 1000, 80, 1000)$	6	$(28, 700, 560, 0, 13)$	1606.66	2	0
1	$(1000, 20, 1000, 80, 1000)$	7	$(21, 21, 71, 0, 32)$	1662.80	3	0
1	$(1000, 20, 1000, 80, 1000)$	8	$(10, 10, 10, 0, 0)$	1722.41	5	0
1	$(1000, 20, 1000, 80, 1000)$	9	$(1, 1, 1, 1, 1)$	1732.23	8	0

Tabla 1: Cálculo de distancias entre instancias de test y de entrenamiento.

Realicemos dos ejecuciones distintas, usando en ambos casos los mismos datasets de los ejemplos anteriores, pero con diferentes valores para el parámetro K (número de vecinos a considerar) del algoritmo KNN.

Ejemplo 4 *Clasificación con algoritmo KNN usando $K = 1$*

En este caso solo se considera la primera instancia más cercana. Puede ver en la tabla 2 la instancia más cercana a cada una de las instancias del dataset de test. Para la instancia 0 del dataset de test (dtC), la instancia más cercana del dataset de entrenamiento (dtM) es la número 8 (a distancia 5.20); tal instancia tiene valor 0 como etiqueta. Para la instancia 1 de dtC, la instancia más cercana de dtM es la número 3 (a distancia 709.60); tal instancia tiene valor 1 como etiqueta. Por tanto, la instancia 0 se clasificaría con etiqueta 0 y la instancia 1 con etiqueta 1.

Dataset dsC		Dataset dsM				
nº	**instancia**	**nº**	**instancia**	**dist.**	**k**	**l**
0	$(10, 10, 5, 1, 1)$	8	$(10, 10, 10, 0, 0)$	5.20	1	0
1	$(1000, 20, 1000, 80, 1000)$	3	$(1163, 26, 1661, 83, 1200)$	709.60	1	1

Tabla 2: Clasificación del dataset de test (K=3).

Ejemplo 5 *Clasificación con KNN usando $K = 3$*

Ahora consideramos las tres instancias más cercanas. Puede ver en la tabla 3 las tres instancias más cercanas a cada una de las instancias del dataset de test. Tanto la instancia 0 como la 1 se clasificarían con etiqueta 0, ya que en ambos casos, 0 es la etiqueta mayoritaría entre las tres instancias más cercanas.

Dataset dsC		Dataset dsM				
nº	**instancia**	**nº**	**instancia**	**dist.**	**k**	**l**
0	$(10, 10, 5, 1, 1)$	8	$(10, 10, 10, 0, 0)$	5.20	1	0
0	$(10, 10, 5, 1, 1)$	0	$(1, 2, 3, 4, 5)$	13.19	2	0
0	$(10, 10, 5, 1, 1)$	9	$(1, 1, 1, 1, 1)$	13.34	3	0
1	$(1000, 20, 1000, 80, 1000)$	3	$(1163, 26, 1661, 83, 1200)$	709.60	1	1
1	$(1000, 20, 1000, 80, 1000)$	6	$(28, 700, 560, 0, 13)$	1606.66	2	0
1	$(1000, 20, 1000, 80, 1000)$	7	$(21, 21, 71, 0, 32)$	1662.80	3	0

Tabla 3: Clasificación del dataset de test (K=3).

Como curiosidad, si comparamos las etiquetas proporcionadas por el clasificador KNN usando $K = 1$ (ejemplo 4) y las que tenía el dataset de test original, podemos ver que se obtiene una tasa de acierto del 100 %. En el caso de usar $K = 3$ (ejemplo 5) podemos ver que se obtiene una tasa de acierto del 50 %.

5. Práctica a entregar

5.1. Finalidad del programa

El propósito del programa Fraud4 es clasificar las instancias de un dataset de test. Para ello, el programa leerá dos ficheros dataset, un fichero con el dataset modelo y un fichero con el dataset de test (dataset a clasificar). El programa debe crear un nuevo fichero dataset de salida idéntico al dataset de test, pero en el que las etiquetas de cada instancia, posiblemente han cambiado con el proceso de clasificación.

El programa puede recibir varios parámetros por la línea de comandos para definir cómo ejecutar el algoritmo de clasificación. Consulte la sintaxis y ejemplos de ejecución en la sección 5.2.

El programa `Fraud4` llevará a cabo los siguientes pasos:

1. El programa comienza procesando los argumentos opcionales (los que empiezan con el carácter `'-'`) pasados al programa. Estos argumentos deben aparecer al principio de la línea de comandos, aunque pueden hacerlo en cualquier orden. Se debe analizar también si el programa se ejecutó pasando al menos el nombre de los ficheros dataset modelo y de test.

2. A continuación, leeremos el contenido de los ficheros dataset modelo y de test.

3. Clasificaremos el dataset de test con ayuda del dataset modelo.

4. Salvaremos el dataset de test ya clasificado en un fichero.

5.2. Sintaxis y ejemplos de ejecución

Como ya indicamos anteriormente, este programa tiene varios parámetros opcionales que pueden pasarse a través de la línea de comandos. Ta-

les parámetros son los primeros que aparecen, y podrían hacerlo en cualquier orden relativo. Tras los parámetros opcionales aparecen los nombres de dos ficheros `dts`, que corresponden a los datasets modelo y de test respectivamente.

La **sintaxis de ejecución** del programa desde un terminal es:

```
Fraud4> build/Fraud4 [-K1 <K1>] [-K2 <K2>] [-nr] [-o
↪  <classifiedTestDataset.dts>] <trainingDataset.dts> <testDataset.dts>
```

La explicación de cada argumento del programa es la siguiente:

- **-K1** `K1`: valor entero con el número de clústeres a utilizar con el algoritmo de reducción de dimensionalidad, si es que se aplica. Por defecto toma valor 5.
- **-K2** `K2`: valor entero con el número de vecinos a considerar en el algoritmo KNN (6 por defecto).
- **-nr**: Si este parámetro está presente, no se aplica la reducción de dimensionalidad a los datasets modelo y a clasificar. Por defecto, sí se aplica.
- **-o** `classifiedTestDataset.dts`: nombre del fichero donde se guarda el dataset de salida, siendo `tests/output/output.dts` el nombre por defecto si no se usa este parámetro.
- `trainingDataset.dts`: nombre del fichero con el dataset de entrenamiento usado como modelo del algoritmo KNN.
- `testDataset.dts`: nombre del fichero dataset de test. Contiene las instancias a clasificar, para las cuales ignoramos las etiquetas que tenían previamente.

Nota: Los parámetros opcionales (los que comienzan con **-**) se pueden introducir en cualquier orden relativo, pero deben estar al principio, antes de los nombres de los dos ficheros dataset de entrada.

Veamos algunos ejemplos de ejecución del programa desde la línea de comandos:

Ejemplo 6 *Faltan argumentos para la ejecución del programa.*

```
Fraud4> build/Fraud4
```

Debe proporcionarse al menos el nombre del dataset modelo y del dataset de test. La salida del programa será la que genera la función `showHelp()` *incluida en* `main.cpp`*, usando el mensaje* `"Name of training dat aset not provided"`*:*

```
ERROR in Fraud4 parameters. Name of training dataset not provided
Run with the following arguments:
Fraud4 [-K1 <K1>] [-K2 <K2>] [-nr] [-o <classifiedDataset.dts>]
↪  <trainingDataset.dts> <testClassify.dts>

Parameters:
-K1 <K1>: an integer with the number of clusters to use in the reduction
↪  dimensionality algorithm (5 by default)
-K2 <K2>: an integer with the number of neighbouring instances to use in the
↪  KNN algorithm (6 by default)
-nr: Indicates that dimensionality reduction should not be applied in the
↪  clustering algorithm (by default dimensionality reduction is done)
[-o <classifiedDataset.dts>]: name of the file where the classified dataset
↪  will be saved (tests/output/output.dts by default)
<trainingDataset.dts>: name of the dataset file used as model (training
↪  dataset)
<testClassify.dts>: name of the dataset file to be classified (test dataset)
```

Ejemplo 7 *Falta el nombre del fichero dataset de test.*

```
Fraud4> build/Fraud4 ../Datasets/princeton_training.dts
```

Debe proporcionarse al menos el nombre del dataset modelo y del dataset de test. La salida del programa será la que genera la función `showHelp()` *incluida en* `main.cpp`*, usando el mensaje* `"Name of test dataset not provided"`*:*

```
ERROR in Fraud4 parameters. Name of test dataset not provided
Run with the following arguments:
Fraud4 [-K1 <K1>] [-K2 <K2>] [-nr] [-o <classifiedDataset.dts>]
↪  <trainingDataset.dts> <testClassify.dts>

Parameters:
-K1 <K1>: an integer with the number of clusters to use in the reduction
↪  dimensionality algorithm (5 by default)
-K2 <K2>: an integer with the number of neighbouring instances to use in the
↪  KNN algorithm (6 by default)
-nr: Indicates that dimensionality reduction should not be applied in the
↪  clustering algorithm (by default dimensionality reduction is done)
[-o <classifiedDataset.dts>]: name of the file where the classified dataset
↪  will be saved (tests/output/output.dts by default)
<trainingDataset.dts>: name of the dataset file used as model (training
↪  dataset)
<testClassify.dts>: name of the dataset file to be classified (test dataset)
```

Ejemplo 8 *Ejecución con parámetro opcional erróneo:*

```
Fraud4> build/Fraud4 -f ../Datasets/princeton_training
↪  ../Datasets/princeton_test.dts
```

En este caso, el argumento `-f` *no es válido para Fraud4. La salida del programa será la que genera la función* `showHelp()` *incluida en* `main.cpp`*, usando el mensaje* `"Invalid option -f"`*.*

Ejemplo 9 *Clasificación del dataset* `princeton_test.dts` *usando el dataset* `princeton_training.dts` *como modelo.*

```
Fraud4> build/Fraud4 ../Datasets/princeton_training.dts
↪  ../Datasets/princeton_test.dts
```

Esta ejecución clasifica el dataset `princeton_test.dts` *usando el dataset* `princeton_training.dts` *como modelo y guarda el dataset ya clasificado en el fichero* `test/output/output.dts`*. El clasificador ha usado reducción de dimensionalidad, con un clustering con* $K1 = 5$*. El algoritmo KNN ha usado* $K2 = 6$*.*

Ejemplo 10 *Clasificación del dataset* `princeton_test.dts` *usando el dataset* `princeton_training.dts` *como modelo.*

```
Fraud4> build/Fraud4 -nr ../Datasets/princeton_training.dts
↪  ../Datasets/princeton_test.dts
```

Esta ejecución es similar a la del ejemplo anterior, pero ahora no se hace reducción de dimensionalidad

Ejemplo 11 *Clasificación del dataset* `princeton_test.dts` *usando el dataset* `princeton_training.dts` *como modelo.*

```
Fraud4> build/Fraud4 -K1 3 -K2 5 -o /tmp/princeton_test_classified.dts
↪  ../Datasets/princeton_training.dts ../Datasets/princeton_test.dts
```

En este caso, hemos seleccionado valores distintos a los de por defecto para los parámetros $K1$ *y* $K2$*. El fichero de salida ha sido en este caso* `/tmp/princeton_test_classified.dts`*.*

En las carpetas `CodeProjects/Datasets` y `CodeProjects/Datasets/dataP4`, puede encontrar ficheros dataset `dts` con los que podrá hacer diferentes pruebas. Además, en la carpeta `(Fraud4/tests)` se proporcionan una serie de ficheros con extensión `.test` que contienen tests de integración, y que pueden usarse con el script `runTest.sh` [2].

5.3. Módulos del proyecto

A la hora de implementar los nuevos métodos y funciones de esta práctica, fíjese en las cabeceras de tales métodos y funciones en los ficheros `*.h` correspondientes y en los comentarios que les acompañan, pues en ellos están detalladas sus especificaciones.

Nota importante: Se han retirado a propósito todos los `const` y `&` para los parámetros de los nuevos métodos y funciones que aparecen en todas las clases del proyecto. También se ha retirado el calificador `const` que debe aplicarse a algunos métodos (los métodos que no modifican el objeto implícito). El número de argumentos y los tipos han sido establecidos y **no se han de cambiar**.

Se le invita a definir todos los métodos y funciones externas adicionales que estime oportuno para una adecuada modularización del código.

En esta práctica debe tener en cuenta las especificaciones indicadas en los siguientes módulos:

[2] Los detalles de como ejecutar tests de integración de forma automática se encuentran en el guion Fraud0.

- Clase `Location`. En esta clase debe sobrecargar los operadores de entrada (>>) y salida (<<). También debe sobrecargar los operadores += y /=.
- Clase `Clustering`. En esta clase debe sobrecargar los operadores relacionales.
- Clase `VectorLocation`. En esta clase debe sobrecargar los operadores de entrada (>>) y salida (<<), así como los operadores `[]` y +=.
- Clase `VectorInt`. En esta clase debe sobrecargar los operadores de entrada (>>) y salida (<<), así como los operadores `[]`, +=.
- Clase `DataSet`. En esta clase debe sobrecargar los operadores de entrada (>>) y salida (<<), así como el operador `()`. Además, debe añadir la función externa `classify()`, que permite clasificar un dataset, y el método `nearestInstance()` que permite obtener la k-ésima instancia más cercana a otra dada.
- Módulo `main.cpp`. Contiene la función main() cuya finalidad será clasificar un dataset de test (vea la sección 5.1). Teniendo en cuenta los detalles comentados en tal sección y los pasos esbozados en el código proporcionado para esta función (sección 6.6), complete el código de este módulo.

Recuerde además corregir los fallos detectados en la práctica anterior, pues se procede con una nueva evaluación de cada uno de los métodos implementados, por lo que los errores no corregidos volverían a penalizar en esta práctica.

5.4. Entrega de la práctica

La práctica deberá ser entregada en Prado, durante el periodo habilitado en la propia actividad de entrega, y consistirá en un fichero ZIP del proyecto. El nombre del fichero puede ser cualquiera, pero tendrá la extensión zip. Por ejemplo, podría llamarse `PRACTICAMP.zip`. Para obtener este fichero se sugiere utilizar el script `runZipProject.sh` (ver el guion de `Fraud0`). Compruebe que lo entregado es compilable y operativo y, no olvide poner el nombre de los componentes del equipo en la cabecera del fichero `main.cpp`.

Recuerde que debe incluir sus apellidos y nombre (y si procede, el de su pareja de prácticas) en el fichero `main.cpp` de la práctica, incluyendo los siguientes tokens tras el token `@file main.cpp`:

```
@author estudiante1: apellidos*, nombre*
@author estudiante2: apellidos*, nombre* (solo si procede)
```

6. Código para la práctica

6.1. Location.h

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file Location.h
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 6 de octubre de 2025, 12:27
 */

#ifndef LOCATION_H
#define LOCATION_H

#include <iostream>
#include <string>

/**
 * @class Location
 * @brief It stores data about a location where people spend money on some
 * service or product. A location contains data about the x and y position
 * (double values) in the 2D space and the name of the location
 * (a string that can contain several words).
 */
class Location {
public:
    /**
     * @brief Constructor that builds a Location object with a value of 0
     * for the x and y coordinates, and an empty string for the name.
     */
    Location();

    /**
     * @brief Returns the x-coordinate of this location
     * Query method
     * @return The x-coordinate of this location
     */
    double getX() const;

    /**
     * @brief Returns the y-coordinate of this location
     * Query method
     * @return The y-coordinate of this location
     */
    double getY() const;

    /**
     * @brief Returns the name of this location
     * Query method
     * @return The name of this location
     */
    std::string getName() const;

    /**
     * @brief Checks if the position of this object is inside the area
     * determined by the two given locations: Location A (@p bottomLeft) and
     * B (@p topRight).
     *
     *      +------------+(B)
     *      |            |
     *      |            |  +(O3)
     *      |            |
     *      |     +(O2)  |
     *      |            |
     *      |            |
     *   (A)+------------+
     *          +(O1)
     *
     * Examples of returning false:
     * - Object O3 is outside the given area (O3.x > B.x)
     * - Object O1 is outside the given area (O1.y < A.y)
     * Example of returning true:
     * - Object O2 is inside the given area
     * - Object A is inside the given area
     * - Object B is inside the given area
     *
     * Query method.
     * @param bottomLeft The Location of the bottom left point. Input parameter
     * @param topRight The Location of the top right point. Input parameter
     * @return true if the Location of this object is inside the area
     * determined by the two given Locations
     */
    bool isInsideArea(const Location &bottomLeft, const Location &topRight) const;

    /**
     * @brief Calculates the square of the Euclidean distante from this location
     * to the provided location. That is, it loc1 and loc2 are two
     * Location objects, then this method returns:
     * \f$ (loc1.x - loc2.x)^2 + (loc1.y - loc2.y)^2 \f$
```

```
     * Query method
     * @param location A Location object. Input parameter
     * @return Returns the square of the Euclidean distante from this
     * location to the provided location.
     */
    double squaredDistance(const Location& location) const;

    /**
     * @brief Calculates the Euclidean distante from this location to the provided
     * location. That is, it loc1 and loc2 are two Location objects, then
     * this method returns:
     * \f$ \sqrt{ (loc1.x - loc2.x)^2 + (loc1.y - loc2.y)^2 } \f$
     * Note that this method can be implemented using the squaredDistance()
     * method
     * Query method
     * @param location A Location object. Input parameter
     * @return The Euclidean distante from this location to the provided
     * location.
     */
    double distance(const Location& location) const;

    /**
     * @brief Obtains a string with the x and y coordinates
     * and the name (whitespace separated). If the name is an empty string,
     * then only the x and y coordinates are included in the resulting string.
     * To convert the x and y coordinates to a string you must use the
     * std::to_string(int) C++ function.
     * Query method
     * @return string with information about this Location object
     */
    std::string toString() const;

    /**
     * @brief Sets the x-coordinate in this object
     * Modifier method
     * @param x The new value for the x-coordinate. Input parameter
     */
    void setX(double x);

    /**
     * @brief Sets the y-coordinate in this object
     * Modifier method
     * @param y The new value for the y-coordinate. Input parameter
     */
    void setY(double y);

    /**
     * @brief Sets the name in this object
     * Modifier method
     * @param name A string with the new value for the name. Input parameter
     */
    void setName(const std::string& name);

    /**
     * @brief Sets the x and y coordinates, and the the name in this object
     * Modifier method
     * @param x The new value for the x-coordinate. Input parameter
     * @param y The new value for the y-coordinate. Input parameter
     * @param name A string with the new value for the name. Input parameter
     */
    void set(double x, double y, const std::string& name);

    /**
     * @brief Reads from the provided input stream the information
     * to fill this Location object.
     * @note This method reads two double values (space separated) from the
     * input stream that are used to set the x and y coordinates of this object.
     * Then, it reads the string that can be formed from the current position of
     * the input stream to the end of the line. This string is then trimmed
     * (see the Trim(string) function) to remove its spaces at the beginning
     * and at the end. The resulting string is used to set the name of this
     * Location object. Note that the name can have several words separated by
     * whitespaces.
     * @param is Input stream. Input/output parameter
     */
    void load(std::istream is);

       /**
     * @brief Adds to the values of the x and y coordinates of this object,
     * the x and y coordinates of the provided location
     * Modifier method
     * @param location A Location object. Input parameter
     * @return A reference to this object.
     */
    Location operator+=(Location location);

    /**
     * @brief Divides the value of the x and y coordinates of this object by the
     * provided double value.
     * Modifier method
     * @throw std::invalid_argument Throws an std::invalid_argument exception
     * if value==0
     * @param value A double value. Input parameter
     * @return A reference to this object.
     */
    Location operator/=(double value);

private:
    /**
     * The x-coordinate of this location
```

```
     */
    double _x;

    /**
     * The y-coordinate of this location
     */
    double _y;

    /**
     * The name of this location. It can contain several words.
     */
    std::string _name;
}; // end of class Location

/**
 * Removes spaces and \t characters at the beginning and at the end of the
 * provided string @p myString. If the provided string @p myString is empty
 * or contains only spaces or \t characters then @p myString will contain an
 * empty string after calling to this function.
 * @note This function can be easily implemented using the methods
 * find_first_not_of(string) and find_last_not_of(string) of class string.
 * @param myString a string. Input/Output parameter
 */
void Trim(std::string & myString);

/**
 * @brief Overloading of the stream insertion operator for Location class.
 * Sends to the output stream the x and y coordinates (converted to string)
 * and name, separated by a whitespace.
 * The x and y coordinates should be converted to string using the
 * std::to_string() C++ function.
 * @param os The output stream to be used. Input/output parameter
 * @param obj The Location object. Input parameter
 * @return A reference to the output stream
 */
std::ostream operator<<(std::ostream os, Location location);

/**
 * @brief Overloading of the stream extraction operator for Location class.
 * It reads two double values (space separated) from the input stream that
 * are used to set the x and y coordinates of the given Location object. Then,
 * it reads the string that can be formed from the current position of the
 * input stream to the end of the line. This string is then trimmed (see
 * the Trim(string) function) to remove its spaces at the beginning
 * and at the end. The resulting string is used to set the name of the provided
 * location. Note that the name can have several words separated by whitespaces.
 * @param is The input stream to be used. Input/output parameter
 * @param obj The Location object. Output parameter
 * @return A reference to the input stream
 */
std::istream operator>>(std::istream is, Location location);

#endif /* LOCATION_H */
```

6.2. VectorLocation.h

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file VectorLocation.h
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 30 de julio de 2025, 11:27
 */

#ifndef VECTORLOCATION_H
#define VECTORLOCATION_H

#include "Location.h"

/**
 * @class VectorLocation
 * @brief A VectorLocation object contains a vector of Location objects. It has
 * a capacity (maximun number of Locations that can be stored in the vector)
 * and a size (number of Locations the vector currently contains).
 * The public methods of this class do not allow a VectorLocation to contain two
 * Location objects with identical names.
 */
class VectorLocation {
public:
    /**
     * @brief It builds a VectorLocation object (vector of Location objects)
     * with a size and capacity equal to the provided value (@p size).
     * Each element in the vector is initialized with the default Location
     * constructor.
     * @throw std::out_of_range Throws a std::out_of_range exception if
     * @p size < 0
     * @param size The size for the vector of Location in this object. Input
     * parameter
     */
    VectorLocation(int size=0);

    /**
     * @brief Copy constructor
     * @param orig the VectorLocation object used as source for the copy.
     * Input parameter
     */
    VectorLocation(VectorLocation orig);

    /**
     * @brief Destructor
     */
    ~VectorLocation();

    /**
     * @brief Overloading of the assignment operator for VectorLocation class
     * Modifier method
     * @param orig the VectorLocation object used as source for the assignment.
     * Input parameter
     * @return A reference to this object
     */
    VectorLocation operator=(VectorLocation orig);

    /**
     * @brief Gets the number of elements in the vector of this object
     * Query method
     * @return The number of elements
     */
    int getSize() const;

    /**
     * @brief Gets the capacity of the vector in this object
     * Query method
     * @return The capacity of the vector in this object
     */
    int getCapacity() const;

    /**
     * @brief Obtains a string with information about this VectorLocation object,
     * in the following format:
     * - First line, the number of Location objects in this vector, converted to
     * a string (using the to_string(int) C++ function).
     * - For each Location, a line with its information, converted to
     * a string with the Location::toString() method.
     *
     * For example, the following is the content of the string for a
     * VectorLocation object with 4 locations:
4
24.8 14.9 Quadrangle
25.6 14.9 Ivy
26.4 14.9 Cottage
27.3 14.5 Cap & Gown
     * Query method
     * @return string with information about this VectorLocation object
     */
    std::string toString() const;

    /**
```

```
 * @brief Searches the provided Location in the array of locations in this
 * object. If found, it returns the position where it was found. If not,
 * it returns -1. We consider that position 0 is the first location in the
 * list of locations and this->getSize()-1 the last location.
 * In order to find a location consider only equality in the name field.
 * Query method
 * @param location A Location. Input parameter
 * @return If found, it returns the position where the location
 * was found. Otherwise it returns -1
 */
int findLocation(const Location& location) const;

/**
 * @brief Returns a VectorLocation object with those locations whose
 * positions are inside the area determined by the two given Locations.
 * Query method
 * @param bottomLeft The Location of the bottom left point. Input parameter
 * @param topRight The Location of the top right point. Input parameter
 * @return A VectorLocation with the selected Locations.
 */
VectorLocation select(const Location & bottomLeft,
    const Location &topRight) const;

/**
 * @brief Removes all the elements in this object, leaving the container
 * with a size equal to 0. It only needs to set the number of elements
 * (_size field) to zero.
 * Modifier method
 */
void clear();

/**
 * @brief Gets a const reference to the Location element at the given
 * position
 * Query method
 * @throw std::out_of_range Throws an std::out_of_range exception if the
 * given position is not valid.
 * @param pos position in the VectorLocation object. Input parameter
 * @return A const reference to the Location element at the given position
 */
const Location & at(int pos) const;

/**
 * @brief Gets a reference to the Location element at the given position.
 * Modifier method
 * @throw std::out_of_range Throws an std::out_of_range exception if the
 * given position is not valid
 * @param pos position in the VectorLocation object. Input parameter
 * @return A reference to the Location element at the given position.
 */
Location & at(int pos);

/**
 * @brief Appends a copy of the given Location object at the first free
 * position in the array of Location in this object. The location is only
 * appended to this object if it was not already found in this object or
 * its name is an empty string.
 * If the dynamic array of Location was full (its capacity was full), this
 * method automatically reallocates a new array with a capacity equal to
 * the current capacity plus an extra block of size equal to BLOCK_SIZE.
 * Modifier method
 * @param value the new Location object to be appended. Input parameter
 * @return true if the given Location could be inserted in this
 * VectorLocation object; false otherwise (the location was already found
 * in this object)
 */
bool append(const Location& location);

/**
 * @brief Appends to this VectorLocation object, the list of
 * Location objects contained in the provided VectorLocation object
 * that are not found (using VectorLocation::findLocation(Location)) in
 * this object.
 * This method could be implemented with the help of the method
 * VectorLocation::append(const Location & location), to append to this
 * object, the Locations of the provided VectorLocation object.
 * Modifier method
 * @param crimeSet A VectorLocation object. Input parameter
 */
void join(const VectorLocation& locations);

/**
 * Sorts the array of locations in this object by increasing alphabetical
 * order of the name of its location (a string).
 * Modifier method
 */
void sort();

/**
 * @brief Gets the position in this vector of the Location object nearest to
 * the provided location
 * Query method
 * @param location A Location object. Input parameter
 * @return the position of the Location object nearest to the provided
 * location.
 * If returns -1 if this vector is empty
 */
int nearest(Location location);

/**
```

```
     * Assigns the provided value to all the elements in this vector
     * Modifier method
     * @param location A Location object. Input parameter
     */
    void assign(Location location);

    /**
     * @brief Reads from the provided input stream the information
     * to fill this VectorLocation object. See files *.loc in the folder
     * DataSets as examples of this kind of file.
     * @note This method should remove any Location previously contained in the
     * provided VectorLocation object.
     * @note This operator throws an exception in some error cases (see below).
     * Before throwing the corresponding exception, this method clears
     * the object (it calls to clear() method) to leave the object in a
     * consistent state.
     * Modifier method
     * @throw std::out_of_range Throws a std::out_of_range exception if the
     * number of Location read from the input stream is negative.
     * @param is Input stream. Input/output parameter
     */
    void load(std::istream is);

    /**
     * @brief Overloading of the [] operator for VectorLocation class
     * Query method
     * @param index index of the element. Input parameter
     * @return A constant reference to the element at position @p index
     */
    Location operator[](int index);

    /**
     * @brief Overloading of the [] operator for VectorLocation class
     * Modifier method
     * @param index index of the element. Input parameter
     * @return A reference to the element at position @p index
     */
    Location operator[](int index);

    /**
     * @brief Overloads the operator += for the VectorLocation class.
     * It appends to this VectorLocation object a copy of the given Location.
     * The new element is put at the end of the vector.
     * The location is not appended to this object, if it was already found in
     * this object.
     * Take into account, that this method increases the capacity
     * of the internal dynamic array of Location objects if that array is full.
     * Modifier method
     * @param location The Location object to append to this object. Input
     * parameter
     * @return A reference to this object.
     */
    VectorLocation operator+=(Location location);

private:
    /**
     * Pointer to a dynamic array of Locations
     */
    Location* _locations;

    /**
     * Number of Location objects contained in the dynamic array _locations
     */
    int _size;

    /**
     * Maximun number of Location objects that can be stored in the dynamic array
     * _locations
     */
    int _capacity;

    /**
     * Size of new memory blocks when resizing the dynamic array _locations
     */
    static const int BLOCK_SIZE=20;
}; // end of class VectorLocation

/**
 * @brief Overloading of the stream insertion operator for VectorLocation class.
 * It sends to the output stream a line for each Location of the given
 * VectorLocation object. Each line will contain the x and y coordinates of
 * a Location, separated by a whitespace.
 * @param os The output stream to be used. Input/output parameter
 * @param vector The VectorLocation object. Input parameter
 * @return @p os A reference to the output stream
 */
std::ostream operator<<(std::ostream os, VectorLocation vector);

/**
 * @brief Overloading of the stream extraction operator for VectorLocation class.
 * This operator first reads from the input stream a line containing an integer n,
 * which corresponds to the number of Location objects to be read from the input stream.
 * Then, it reads n Location objects from the input stream, storing them in the provided
 * VectorLocation object (@p vector).
 * @note This operator should remove any Location previously contained in the
 * provided VectorLocation @p vector.
 * @note This operator throws an exception in some error cases (see below).
 * Before throwing the corresponding exception, this operator clears
 * the provided object (it calls to clear() method) to leave it in a
 * consistent state.
```

```
 * @throw std::out_of_range Throws a std::out_of_range exception if the number
 * of locations read from the file is negative.
 * @param is The input stream to be used. Input/output parameter
 * @param vector The VectorLocation object to be filled. Input/output parameter
 * @return @p is A reference to the input stream
 */
std::istream operator>>(std::istream is, VectorLocation vector);

#endif /* VECTORLOCATION_H */
```

6.3. VectorInt.h

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file VectorInt.h
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 30 de julio de 2025, 9:54
 */

#ifndef VECTORINT_H
#define VECTORINT_H

#include <iostream>
#include <string>

/**
 * @class VectorInt
 * @brief A VectorInt object contains a vector of integer elements. It has
 * a capacity (maximun number of elements that can be stored in the vector)
 * and a size (number of elements the vector currently contains).
 */
class VectorInt {
public:

    /**
     * @brief It builds a VectorInt object (vector of integers) with a
     * size and capacity equal to the provided value (@p size). Each element
     * will be filled with a value equal to 0.
     * @throw std::out_of_range Throws a std::out_of_range exception if
     * @p size < 0
     * @param size The size for the vector of integers in this object. Input
     * parameter
     */
    VectorInt(int size=0);

    /**
     * @brief Copy constructor
     * @param orig the VectorInt object used as source for the copy. Input
     * parameter
     */
    VectorInt(VectorInt orig);

    /**
     * @brief Destructor
     */
    ~VectorInt();

    /**
     * @brief Overloading of the assignment operator for VectorInt class
     * Modifier method
     * @param orig the VectorInt object used as source for the assignment. Input
     * parameter
     * @return A reference to this object
     */
    VectorInt operator=(VectorInt orig);

    /**
     * @brief Gets the number of elements in the vector of this object
     * Query method
     * @return The number of elements
     */
    int getSize();

    /**
     * @brief Gets the capacity of the vector in this object
     * Query method
     * @return The capacity of the vector in this object
     */
    int getCapacity();

    /**
     * @brief Compares the integer vectors of this object and the provided
     * object, and returns the number of identical elements in both of them. The
     * comparison is performed in order, element by element, in both vectors.
     * For example, given the following two vectors:
     * 2 1 2 3 5
     * 1 1 2 4 5
     * this method will return 3 (there is a match in positions 1, 2 and 4)
     * @throw std::invalid_argument Throws an std::invalid_argument exception
     * if the sizes of this and the provided object are different
     * Query method
     * @param other A VectorInt object. Input parameter
     * @return The number of identical elements in the vectors of this
     * and the provided object.
     */
    int countIdenticalElements(VectorInt other);

    /**
     * @brief Obtains a string with information about this VectorInt object,
     * in the following format:
     * - First line, the number of elements in this vector.
     * - Second line, the elements in this vector, separated by a whitespace.
```

```
     * Take into account that a '\n' should appear after the last element
     * instead of a whitespace.
     * Query method
     * @return string with information about this VectorInt object
     */
    std::string toString();

    /**
     * @brief Gets the Euclidean distance between this and the provided object.
     * The Euclidean distance between two points \f$ P=(p_1, p_2, ... , p_n) \f$
     * and \f$ Q=(q_1, q_2, ... , q_n) \f$ in an n-dimensional space \f$ R^n \f$
     * is the length of the line segment connecting them, calculated as the
     * square root of the sum of the squared differences of their components:
     * \f$ d(P,Q)=\sqrt{ \sum_{i=1}^{n} (p_i-q_i)^{2} } \f$
     * @throw std::invalid_argument Throws an std::invalid_argument exception
     * if the size of this object and the provided one are not equal
     * @throw std::invalid_argument Throws an std::invalid_argument exception
     * if the size of the provided object is zero
     * Query method
     * @param other A VectorInt. Input parameter
     * @return The Euclidean distance between this and the provided objects
     */
    double distance(VectorInt other);

    /**
     * @brief Assigns the provided value to all the elements in this vector
     * Modifier method
     * @param value An integer value. Input parameter
     */
    void assign(int value=0);

    /**
     * @brief Appends the given integer value at the end (first free position)
     * of the array of integers in this object.
     * If the dynamic array of integers was full (its capacity was full), this
     * method automatically reallocates a new array with a capacity equal to
     * the current capacity plus an extra block of size equal to BLOCK_SIZE.
     * Modifier method
     * @param value the new integer value to be appended. Input parameter
     */
    void append(int value);

    /**
     * @brief Removes all the elements in this object, leaving the container
     * with a size equal to 0. It only need to set the number of elements
     * (_size field) to zero.
     * Modifier method
     */
    void clear();

    /**
     * @brief Gets a const reference to the integer element at the given
     * position
     * Query method
     * @param pos position in the VectorInt object. Input parameter
     * @throw std::out_of_range Throws an std::out_of_range exception if the
     * given position is not valid.
     * @return A const reference to the integer element at the given position
     */
    int at(int pos);

    /**
     * @brief Gets a reference to the integer element at the given position.
     * Modifier method
     * @param pos position in the VectorInt object. Input parameter
     * @throw std::out_of_range Throws an std::out_of_range exception if the
     * given position is not valid
     * @return A reference to the integer element at the given position.
     */
    int at(int pos);

    /**
     * @brief Overloading of the [] operator for VectorInt class
     * Query method
     * @param index index of the element. Input parameter
     * @return A constant reference to the element at position @p index
     */
    int operator[](int index);

    /**
     * @brief Overloading of the [] operator for VectorInt class
     * Modifier method
     * @param index index of the element. Input parameter
     * @return A reference to the element at position @p index
     */
    int operator[](int index);

    /**
     * @brief Overloads the operator += for the VectorInt class.
     * It appends to this VectorInt object the given integer value. The
     * new element is put at the end.
     * Take into account, that this method increases the capacity
     * of the internal dynamic array of integers if that array is full.
     * Modifier method
     * @param value The integer value to append to this object. Input parameter
     * @return A reference to this object.
     */
    VectorInt operator+=(int value);

private:
```

```
    /**
     * Pointer to a dynamic array of integers
     */
    int* _values;

    /**
     * Number of elements contained in the dynamic array _values
     */
    int _size;

    /**
     * Maximun number of elements that can be stored in the dynamic array
     * _values
     */
    int _capacity;

    /**
     * Size of new memory blocks when resizing the dynamic array _values
     */
    static const int BLOCK_SIZE=20;

}; // end of class VectorInt

/**
 * @brief Overloading of the stream insertion operator for VectorInt class.
 * It sends to the output the information about the provided VectorInt
 * object in the following format:
 * - First line, the number of elements in this vector.
 * - Second line, the elements in the provided vector, separated by a whitespace.
 * Take into account that a '\n' should appear after the last element
 * instead of a whitespace.
 * @param os The output stream to be used. Input/output parameter
 * @param vector The VectorInt object. Input parameter
 * @return os A reference to the output stream
 */
std::ostream operator<<(std::ostream os, VectorInt vector);

/**
 * @brief Overloading of the stream extraction operator for VectorInt class.
 * @note This operator should remove any previously information contained in the
 * provided VectorInt @p vector.
 * @note This operator throws an exception in some error cases (see below).
 * Before throwing the corresponding exception, this operator clears
 * the provided object (it calls to clear() method) to leave it in a
 * consistent state.
 * @throw std::out_of_range Throws a std::out_of_range exception if the number
 * of elements read from the file is negative.
 * @param is The input stream to be used. Input/output parameter
 * @param vector The VectorInt object to be filled. Input/output parameter
 * @return A reference to the input stream
 */
std::istream operator>>(std::istream is, VectorInt vector);

#endif /* VECTORINT_H */
```

6.4. Clustering.h

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file Clustering.h
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 22 de octubre de 2025, 11:57
 */

#ifndef CLUSTERING_H
#define CLUSTERING_H

#include "VectorInt.h"
#include "VectorLocation.h"

/**
 * @class Clustering
 * @brief This class is used to obtain a clustering of a given set of locations
 * (Location objects). A clustering algorithm groups the input locations
 * into K disjoint groups (clusters). The clustering algorithm used in this
 * class is the K-Means algorithm. Each group will consist of a disjunct subset
 * of the original location set and a representative location (centroid), which
 * is calculated as the vector of the means of the locations in the group. The
 * run() method in this class executes the algorithm. An example of use is the
 * following:
VectorLocation locations;
// Suppose here code to fill the vector of locations
Clustering clustering;
clustering.set(locations, 5);
clustering.run();
 */
class Clustering {
public:
    /**
     * @brief Constructor that builds a Clustering object initializing
     * the field (_K) for the number of clusters with 0; the field (_isDone)
     * that tells whether the run() method has already been executed
     * with false; the fields _sumWCV and _numIterations with 0 and the field
     * _seed with DEFAULT_RANDOM_SEED.
     * The fields (_locations, _clusters, _centroids) are initialized with
     * the default constructor of their classes.
     */
    Clustering();

    /**
     * @brief Gets the number of clusters
     * Query method
     * @return The number of clusters
     */
    int getK();

    /**
     * @brief Gets the current vector of centroids
     * Query method
     * @return A const reference to the vector of centroids (a VectorLocation
     * object).
     */
    VectorLocation getCentroids();

    /**
     * @brief Indicates whether the clustering algorithm (run() method) has
     * already been executed for this Clustering object.
     * Query method
     * @return true if the run() method has already been executed for this
     * Clustering object; false otherwise
     */
    bool isDone();

    /**
     * @brief Gets the number of locations in the set of locations of this
     * Clustering object.
     * Query method
     * @return The number of locations in the set of locations of this
     * Clustering
     */
    int getNumLocations();

    /**
     * @brief Gets the cluster number for the Location determined by the
     * provided integer (@p locationIndex)
     * Query method
     * @param locationIndex An integer that determines the position of a
     * location in the vector of Location objects. Input parameter
     * @return The cluster number for the Location determined by
     * the provided integer (@p locationIndex). If the
     * clustering algorithm has not been run (_isDone is false), then it returns
     * -1.
     */
    int clusterOf(int locationIndex);

    /**
     * @brief Gets the value of the sum of the within-cluster variances of this
```

```
     * Clustering. This value is obtained from the field _sumWCV
     * @note The returned value will likely be a garbage value if the run()
     * method has not yet been executed (_isDone is false)
     * Query method
     * @return A double with the sum of within-cluster variances of this
     * Clustering
     */
    double getSumWCV();

    /**
     * @brief  Gets the number of iterations used in the KMeans
     * algorithm until no cluster assignment change occurs.
     * It obtains the number of iterations (field _numberIterations) used
     * in the KMeans algorithm until no cluster change occurred.
     * @note The returned value will likely be a garbage value if the run()
     * method has not yet been executed (_isDone is false)
     * Query method
     * @return The number of iterations used in the KMeans algorithm
     */
    int getNumIterations();

    /**
     * @brief  Gets a string with information about the provided cluster (index
     * of a cluster). That string contains a line for each location in the given
     * cluster. The information of each location consists of an integer with
     * its location index (whithin the _location field), a whitespace, and
     * the information returned by the Location::toString() method for that
     * location. An example is the following (a cluster with three locations):
0 3.300000 9.100000 Graduate College
6 12.600000 17.500000 Admission Information Center
7 12.500000 20.000000 Rockefeller College
     * Query method
     * @param cluster An integer, that defines the index of a cluster. Input
     * parameter
     * @return Gets a string with information about the provided cluster. If the
     * clustering algorithm has not been run (_isDone is false) or an
     * invalid value of cluster is provide, then it returns an empty string.
     */
    std::string clusterInfo(int cluster);

    /**
     * @brief Obtains a string with the some statistics data about this
     * Clustering object.
     * The first line contains the value of K; the second line contains the
     * value of the sum of within-cluster variances; the third line contains
     * the number of iterations used in the KMeans algorithm.
     * Query method
     * @return A string with the content described above
     */
    std::string getStatistics();

    /**
     * @brief Indicates whether this Clustering object is equivalent to the
     * provided Clustering object (@p other). Two Clustering objects are
     * considered equivalent if they have the same K value,
     * the same sum of within-cluster variances and the same number of
     * iterations in the KMeans algorithm.
     * If either this Clustering object or the provided Clustering object
     * (@p other) has not yet run the clustering algorithm (the run() method),
     * then they are not considered equivalent.
     * Query method
     * @param other A Clustering object. Input parameter
     * @return true if this Clustering object is equivalent to the provided
     * Clustering object (@p other); false otherwise
     */
    bool isEquivalentTo(Clustering other);

    /**
     * @brief Obtains a string with the fields of this Clustering object.
     * The first line contains the value of K; the second line contains the
     * value of the sum of within-cluster variances; the third line contains
     * the number of iterations used in the KMeans algorithm.
     * In the next three lines, information about the cluster number for each
     * location.
     * If follows information about each centroid
     * Next, for each cluster, information about its locations
     * Here, an example is shown:
K=5
Sum of within-cluster variances: 161.136889
Number of iterations: 5
Cluster number for each location:
21
0 2 2 2 2 2 1 1 1 1 1 4 4 4 4 4 4 4 4 4 3
Centroids:
5
3.300000 9.100000
14.880000 21.280000
12.600000 9.460000
18.500000 6.700000
26.088889 15.366667
Cluster 0 information:
0 3.300000 9.100000 Graduate College
Cluster 1 information:
6 12.600000 17.500000 Admission Information Center
7 12.500000 20.000000 Rockefeller College
8 13.700000 22.800000 Art Museum Store
9 17.000000 23.300000 Princeton University Store
10 18.600000 22.800000 Labyrinth Books
Cluster 2 information:
1 11.800000 6.900000 Wawa Public Restrooms
```

```
2 12.600000 7.300000 Princeton Station
3 10.100000 9.700000 Forbes College
4 13.300000 12.200000 Dinky Bar & Kitchen
5 15.200000 11.200000 Whitman College
Cluster 3 information:
20 18.500000 6.700000 Yeh College
Cluster 4 information:
11 20.900000 15.400000 Frist Campus Center
12 23.700000 14.800000 Cannon Dial Elm
13 24.800000 14.900000 Quadrangle
14 25.600000 14.900000 Ivy
15 26.400000 14.900000 Cottage
16 27.300000 14.500000 Cap & Gown
17 28.200000 15.000000 Cloister
18 29.100000 15.200000 Charter
19 28.800000 18.700000 Engineering Quadrangle
     * Query method
     * @return A string with the content described above
     */
    std::string toString();

    /**
     * @brief Sets the vector of locations (_locations), the value of K (_K)
     * and the seed (_seed) with the provided values.
     * The fields _isDone, _clusters and _centroids_ are also initialized
     * appropriately: _isDone is set to false; _clusters is initialized with
     * a VectorInt object with size equal to the number of locations;
     * _centroids is initialized with a VectorLocation object with size K;
     * the fields _sumWCV and _numIterations are set to 0.
     * Modifier method
     * @throw std::invalid_argument Throws an std::invalid_argument exception
     * if K<=0
     * @param locations The vector of locations to used in the clustering
     * algorithm. Input parameter
     * @param K The parameter K for the K-Means algorithm. Input parameter
     * @param seed The seed used to initialize the random number generator
     * (with srand(seed)) in the initialClusterAssignment() method.
     * Input parameter
     */
    void set(VectorLocation locations, int K,
        unsigned int seed=DEFAULT_RANDOM_SEED);

    /**
     * @brief Run the clustering algorithm. The clustering algorithm implemented
     * in this method is the KMeans algorithm.
     * After running this method the following postcondition should be verified:
     * @post The _centroids field should contain the vector of the means of
     * locations in each group
     * @post _clusters (a VectorInt object) should contain the cluster number
     * for each location in the vector of locations (_locations)
     * @post The _isDone field should be set to true
     * @post The _sumWCV field should be set to the sum of the within-cluster
     * variances using the claculateSumWithinClusterVariances() method.
     * @post The _numberIterations field should be set to the number of
     * iterations used by the KMeans algorithm.
     *
     * A brief description of the KMeans algorithm is the following:
     *
     * -# Perform an initial assignment of a cluster to each of the Location
     * objects. This must be done by initializing the random number
     * generator using srand(seed);
     * Then, for each location i, assign cluster number rand() % _K
     * to that location. Note that this step is done by modifying the _clusters
     * field.
     * -# Repeat while any location changes its cluster number:
     *     - For each cluster, recalculate its centroid (update _centroids
     *       field).
     *     - Assign each location to a cluster (update _clusters field).
     *       Each location is assigned to the cluster that provides
     *       the shortest distance to its centroid.
     *
     * Modifier method
     */
    void run();

private:
    /**
     * Default random seed used to initialize the random number
     * generator used in the initialClusterAssignment() method.
     */
    static const unsigned int DEFAULT_RANDOM_SEED=1761560597U;

    /**
     * Vector of Location objects. This will be the input for the
     * clustering algorithm
     */
    VectorLocation _locations;

    /**
     * Number of clusters
     */
    int _K;

    /**
     * Vector of n integers (number of elements in _locations). This will
     * be the output of the clustering algorithm.
     * The integers in this vector define the cluster number (values in the
     * interval [0 - _K-1]) for each one of the Location object in _locations
     */
    VectorInt _clusters;
```

```
    /**
     * A vector of @p _K Location objects with the centroids of each cluster.
     */
    VectorLocation _centroids;

    /**
     * It indicates whether the run() method has already been executed to
     * obtain a clustering of the set of Location objects. The run() method
     * should set this field to true when it ends its execution.
     */
    bool _isDone;

    /**
     * Seed used to initialize the random number generator (with srand(seed)) in
     * the initialClusterAssignment() method.
     */
    unsigned int _seed;

    /**
     * This is the sum of the whithin-cluster variances of the clusters in
     * this Clustering object.
     * The value for this field should be set at the end of run() method
     * with the method calculateSumWCV(). Before running the
     * run() method, this field will likely contain a garbage value.
     * @note The value of the sum of the within-cluster variances can also be
     * retrieved using the method calculateSumWCV().
     * This field is introduced in this class to calculate its value once,
     * and thus avoiding having to recalculate it every time it
     * is needed
     */
    double _sumWCV;

    /**
     * This field represents the number of iterations used in the KMeans
     * algorithm until no cluster change occurred. Before running the
     * run() method, this field will likely contain a garbage value.
     * The value for this field should be set at the end of run() method.
     */
    int _numIterations;

    /**
     * @brief Performs an initial assignment of a cluster to each one of the
     * Location objects. This method begins by initializing the random number
     * generator using:
     * srand(seed);
     * Then, for each location i, assign cluster number rand() % _K to that
     * location.
     * Modifier method
     */
    void initialClusterAssignment();

    /**
     * @brief Calculates the sum of the within-cluster variances of this
     * Clustering object.
     * The within-cluster variance of a cluster C_j is calcultated with:
     * $\sum_{p_i \in C_j} squaredDistance(p_i, centroid_{C_j})$
     * @return A double with the sum of within-cluster variances of this
     * Clustering
     * Query method
     */
    double calculateSumWCV();
}; // end of class Clustering

/**
 * @brief Overloading of the less-than operator for Clustering class
 * @param clustering1 A Clustering object. Input parameter
 * @param clustering2 A Clustering object. Input parameter
 * @return true if the sum of within-cluster variances (WCVs) of
 * @p clustering1 is less than that of @p clustering2 or if both WCVs are
 * equal and the number of iterations of @p clustering1 is less than that of
 * @p clustering2; false otherwise
 */
bool operator<(Clustering clustering1, Clustering clustering2);

/**
 * @brief Overloading of the greater-than operator for Clustering class
 * @param clustering1 A Clustering object. Input parameter
 * @param clustering2 A Clustering object. Input parameter
 * @return true if clustering1 > clustering2; false otherwise
 */
bool operator>(Clustering clustering1, Clustering clustering2);

/**
 * @brief Overloading of the equality operator for Clustering class
 * @param clustering1 A Clustering object. Input parameter
 * @param clustering2 A Clustering object. Input parameter
 * @return true if clustering1 == clustering2; false otherwise
 */
bool operator==(Clustering clustering1, Clustering clustering2);

/**
 * @brief Overloading of the not-equal operator for Clustering class
 * @param clustering1 A Clustering object. Input parameter
 * @param clustering2 A Clustering object. Input parameter
 * @return true if clustering1 != clustering2; false otherwise
 */
bool operator!=(Clustering clustering1, Clustering clustering2);

/**
```

```
 * @brief Overloading of the less-than-or-equal operator for Clustering class
 * @param clustering1 A Clustering object. Input parameter
 * @param clustering2 A Clustering object. Input parameter
 * @return true if clustering1 <= clustering2; false otherwise
 */
bool operator<=(Clustering clustering1, Clustering clustering2);

/**
 * @brief Overloading of the greater-than-or-equal operator for Clustering class
 * @param clustering1 A Clustering object. Input parameter
 * @param clustering2 A Clustering object. Input parameter
 * @return true if clustering1 >= clustering2; false otherwise
 */
bool operator>=(Clustering clustering1, Clustering clustering2);
#endif /* CLUSTERING_H */
```

6.5. DataSet.h

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file DataSet.h
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 30 de julio de 2025, 10:00
 */

#ifndef DATASET_H
#define DATASET_H

#include <string>
#include <iostream>
#include <fstream>

#include "VectorInt.h"
#include "VectorLocation.h"
#include "Clustering.h"

/**
 * @class DataSet
 * @brief A DataSet is an object that contains a set of n instances. Each
 * instance contains a vector of m integers that represents the amount of
 * money spent by a person in each one of m locations.
 * This class uses a bidimensional matrix with n rows and m columns to store
 * the values of the set of instances.
 * This class also contains a vector of n labels (VectorInt _labels) to store the
 * label of each instance. Each label is an integer that defines the class
 * value of the corresponding instance.
 * Finally, this class contains a vector of m Location objects
 * (VectorLocation _locations) to store each locations in the DataSet.
 * In the 2D matrix, each column i is associated with location i in the
 * _locations vector.
 */
class DataSet {
public:
    /**
     * @brief It builds a DataSet object with the provided number of instances
     * and number of locations. This class uses a bidimensional matrix with
     * nInstances rows and nLocations columns to store the values of the set
     * of instances.
     * Note that this constructor also builds the vector (with a size equal to
     * nInstances) of labels and the vector of locations (with a size equal to
     * nLocations).
     * This constructor initializes with a value of 0 all the values in the 2D
     * matrix.
     * The vector of labels is initialized with a value of 0 for each instance.
     * The Location objects in the vector of locations are built with the
     * Location default constructor.
     * @param nInstances An integer with the number of instances.
     * Input parameter
     * @param nLocations An integer with the number of locations.
     * Input parameter
     */
    DataSet(int nInstances = 0, int nLocations = 0);

    /**
     * @brief Copy constructor
     * @param orig the DataSet object used as source for the copy.
     * Input parameter
     */
    DataSet(DataSet orig);

    /**
     * @brief Destructor
     */
    ~DataSet();

    /**
     * @brief Overloading of the assignment operator for DataSet class
     * Modifier method
     * @param orig the DataSet object used as source for the assignment.
     * Input parameter
     * @return A reference to this object
     */
    DataSet operator=(DataSet orig);

    /**
     * @brief Gets the number of instances in this DataSet
     * Query method
     * @return The number of instances in this DataSet
     */
    int getNumInstances();

    /**
     * @brief Gets the number of localizations in this DataSet
     * Query method
     * @return The number of localizations in this DataSet
     */
    int getNumLocations();
```

```
/**
 * @brief Gets the value for the instance instanceIndex at the localization
 * locationIndex.
 * @throw std::out_of_range Throws a std::out_of_range exception if
 * instanceIndex is not a valid index for an instance in this DataSet.
 * @throw std::out_of_range Throws a std::out_of_range exception if
 * locationIndex is not a valid index for an location in this DataSet.
 * Query method
 * @param instanceIndex An integer with the index of the selected instance.
 * Input parameter
 * @param locationIndex An integer with the index of the selected location.
 * Input parameter
 * @return The value for the instance instanceIndex at the localization
 * locationIndex
 */
int getValue(int instanceIndex, int locationIndex);

/**
 * @brief Gets the label (integer value) of the instance at the provided
 * position (instanceIndex)
 * @throw std::out_of_range Throws a std::out_of_range exception if
 * instanceIndex is not a valid index for an instance in this DataSet.
 * Query method
 * @param instanceIndex An integer with the index of the selected instance.
 * Input parameter
 * @return The label (integer value) of the instance at the provided
 * position
 */
int getLabel(int instanceIndex);

/**
 * @brief Gets a const reference to the vector of Location objects in this
 * DataSet
 * Query method
 * @return A const reference to the vector of Location objects in this
 * DataSet
 */
VectorLocation getVectorLocation();

/**
 * @brief Gets a const reference to the vector of labels in this DataSet
 * Query method
 * @return A const reference to the vector of labels in this DataSet
 */
VectorInt getVectorLabels();

/**
 * @brief Obtains a string with information about this DataSet object,
 * in the following format:
 * -Information about the set of locations of this DataSet:
 *    - A line with the number of locations
 *    - For each location, a line with its information, converted to
 *      a string with the Location::toString() method.
 * - Information about the labels of each instance in this DataSet:
 *    - A line with the number of instances
 *    - A line with the labels (space separated) of the set of instances.
 * - Instances of the Dataset. This is the information contained in the
 *   2D matrix of this DataSet.
 *     - For each instance, a line with the values (space separated) of
 *       that instance.
 * Query method
 * @return string with information about this CrimeSet object
 */
std::string toString() ;

/**
 * @brief Sets a new value for the instance instanceIndex at the
 * localization locationIndex.
 * Modifier method
 * @param instanceIndex An integer with the index of the selected instance.
 * Input parameter
 * @param locationIndex An integer with the index of the selected location.
 * Input parameter
 * @param value An integer with the new value for the selected instance.
 * Input parameter
 */
void setValue(int instanceIndex, int locationIndex, int value);

/**
 * @brief Sets a new the label for the provided instance.
 * Modifier method
 * @param instanceIndex An integer with the index of the selected instance.
 * Input parameter
 * @param label An integer with the new value for the label of the selected
 * instance.
 * Input parameter
 */
void setLabel(int instanceIndex, int label);

/**
 * Assigns the provided value to each instance and location. That is, all
 * instances of this DataSet are initialized with the provided value.
 * Modifier method
 * @param value An integer with the value to assign to every instance and
 * location
 * Input parameter
 */
void initInstances(int value = 0);

/**
```

```
 * @brief Removes all the information in this DataSet object:
 * - The vector of Location objects (_locations) is left empty. That is,
 * it should represent a vector with 0 locations.
 * - The vector of labels (_labels) is left empty. That is, it should
 * represent a vector with 0 labels.
 * - The matrix (_values) with the values for each instance is left empty.
 * That is, it should represent a matrix with 0 instances and 0 locations.
 * - The remaining fields of this object are updated accordingly.
 * Modifier method
 */
void clear();

/**
 * @brief Saves this DataSet object in the given file. See files *.dts in
 * the folder DataSets as example of files with DataSet information.
 * Note that the format of the output file is the same as the string that
 * returns the toString() method, except that the save() method should
 * insert the magic string at the beginning of the output file.
 * @throw std::ios_base::failure Throws a std::ios_base::failure exception
 * if the given file cannot be opened or if an error occurs while writing
 * to the file.
 * Query method
 * @param fileName The name of the file where the DataSet will be saved.
 * Input parameter
 */
void save(std::string fileName);

/**
 * @brief Loads into this object the DataSet information stored in the
 * given file. See files *.dts in the folder DataSets as example of files
 * with DataSet information.
 * @note Note that this method should remove any previous information that
 * this object previously contained.
 * @note This method throws an exception in some error cases (see below).
 * Before throwing the corresponding exception, this method clears
 * the object (it calls to clear() method) to leave the object in a
 * consistent state.
 * @throw throw std::invalid_argument Throws a std::invalid_argument
 * exception if an invalid magic string is found in the given file
 * @throw std::out_of_range Throws a std::out_of_range exception if the
 * number of locations or the number of instances in the given file is
 * negative.
 * Modifier method
 * @param fileName The name of the file where the DataSet is stored.
 * Input parameter
 * @throw std::ios_base::failure Throws a std::ios_base::failure exception
 * if the given file cannot be opened or if an error occurs while reading
 * from the file.
 */
void load(std::string fileName);

/**
 * @brief Gets a new DataSet from this DataSet. The DataSet will contain
 * the same number of instances as this DataSet, but the number of locations
 * is given by the number of clusters in the provided Clustering object.
 * Therefore, the returned DataSet will have a reduced number of columns
 * and an equal number of rows compared to this DataSet.
 * The provided Clustering object defines how to reduce the set of columns.
 * Column 0 will contain the sum of the columns of this DataSet belonging
 * to cluster 0, column 1 the sum of the columns belonging to cluster 1,
 * and so on.
 * The vector of locations will be a copy of the vector of centroids of the
 * provided Clustering object.
 * The vector of labels will be a copy of the vector of labels in this
 * DataSet.
 * @throw throw std::invalid_argument Throws a std::invalid_argument
 * exception if the method Clustering::isDone() return false with the
 * provided Clustering object.
 * @throw throw std::invalid_argument Throws a std::invalid_argument
 * exception if the number of locations in the provided Clustering object
 * is not equal to the number of locations in this DataSet.
 * Query method
 * @param clustering A Clustering object.
 * Input parameter
 * @return A reduced DataSet
 */
DataSet getReducedDataSet(Clustering clustering);

/**
 * Returns the index (position) of the nearest instance in this DataSet to
 * the provided instance (@p instance). To do so, the Euclidean distance
 * is used to calculate the distance between two instances.
 * If an instance i has already been selected (selected[i] == true) then
 * it is not considered as a possible nearest instance.
 * @throw std::invalid_argument Throws a std::invalid_argument exception
 * if the size of the provided instance and the number of locations in this
 * Dataset is not equal.
 * Query method
 * @param instance A VectorInt that represents the instance for which
 * we want to calculate the nearest instance.
 * Input parameter
 * @param selected array of bool that defines the already selected instances.
 * This array should have at least a capacity equal to the number of
 * instances in this DataSet.
 * Input parameter
 * @return An integer with the index (position) of the nearest instance to
 * @p instance. It returns -1 if this DataSet does not contain any instance.
 */
int nearestInstance(VectorInt instance, bool selected[]);
```

```
    /**
     * @brief Overloading of the () operator to access to the value at a
     * given position in this DataSet
     * Query method
     * @param instanceIndex number of instance in this DataSet.
     * Input parameter
     * @param locationIndex number of location in this DataSet
     * Input parameter
     * @return A const reference to the value at the given position
     */
    int operator()(int instanceIndex, int locationIndex);

    /**
     * @brief Overloading of the () operator to access to the value at a
     * given position in this DataSet
     * Modifier method
     * @param instanceIndex number of instance in this DataSet.
     * Input parameter
     * @param locationIndex number of location in this DataSet.
     * Input parameter
     * @return A reference to the value at the given position
     */
    int operator()(int instanceIndex, int locationIndex);

private:
    /**
     * A const string with the magic string for text files
     */
    static const std::string MAGIC_STRING_T;

    /**
     * Bidimensional matrix with the values of the instances of this DataSet.
     * It is a 2D matrix of integer values, with n rows and m columns. n is
     * the number of instances in the DataSet and m is the number of locations.
     */
    int** _values;

    /**
     * Number of rows in the 2D matrix. It is the number of instances of the
     * DataSet
     */
    int _nInstances;

    /**
     * Number of columns of the 2D matrix. It is the number of
     * locations of the DataSet
     */
    int _nLocations;

    /**
     * A VectorInt (vector of integers) with the labels of each one of the
     * n instances in this DataSet. The order of the integers in this vector
     * is assumed to correspond to the order of the rows in the 2D matrix of
     * this DataSet.
     */
    VectorInt _labels;

    /**
     * A VectorLocation object containing the m locations used in this DataSet.
     * The order of the locations in this vector is assumed to correspond to
     * the order of the columns in the 2D matrix of this DataSet.
     */
    VectorLocation _locations;
}; // end of class DataSet

/**
 * @brief Overloading of the stream insertion operator for DataSet class.
 * @param os The output stream to be used. Input/output parameter
 * @param dataset The DataSet object. Input parameter
 * @return A reference to the output stream
 */
std::ostream operator<<(std::ostream os, DataSet dataset);

/**
 * @brief Overloading of the stream extraction operator for DataSet class.
 * @note Note that this operator should begin by removing any previous
 * information that the provided object previously contained (a call to clear()
 * method).
 * @throw std::out_of_range Throws a std::out_of_range exception if the number
 * of locations read from the file is negative.
 * @throw std::out_of_range Throws a std::out_of_range exception if the
 * number of locations or the number of instances in the provided input stream
 * is negative.
 * @param is The input stream to be used. Input/output parameter
 * @param dataset The DataSet object to be filled. Input/output parameter
 * @return A reference to the input stream
 */
std::istream operator>>(std::istream is, DataSet dataset);

/**
 * Classifies the instances of the dataset @p dataSetToClassify with
 * the KNN algorithm and using the dataset @p dataSetModel as model to classify.
 * Each instance in @p dataSetToClassify will be classified by changing its
 * label to the one found by the KNN algorithm.
 * @throw std::invalid_argument Throws a std::invalid_argument exception if the
 * number of locations in datasetModel and datasetToClassify is not equal.
 * @throw std::invalid_argument Throws a std::invalid_argument exception
 * if @p doReductionDimensionality is true and @p K1 is less or equals than 0.
 * @throw std::invalid_argument Throws a std::invalid_argument exception
 * if @p K2 is greater than the number of instances in @p datasetModel
```

```
 * @throw std::invalid_argument Throws a std::invalid_argument exception
 * if @p K2 is less than 1
 * @param dataSetModel DataSet used as model to classify. Input parameter
 * @param dataSetToClassify DataSet to be classified. Input/output parameter
 * @param K1 The K value to be used in the clustering algorithm (KMeans) of
 * the dimensionality reduction step. Input parameter
 * @param K2 The K value to be used in the KNN algorithm (to define the number K
 * of nearest neighbours). Input parameter
 * @param doReductionDimensionality Indicates whether or not dimensionality
 * reduction should be applied using the clustering algorithm. Input parameter
 */
void classify(DataSet datasetModel, DataSet datasetToClassify,
        int K1, int K2, bool doReductionDimensionality);

#endif /* DATASET_H */
```

6.6. main.cpp

```
/*
 * Metodología de la Programación
 * Curso 2025/2026
 */

/**
 * @file main.cpp
 * @author Silvia Acid Carrillo <acid@decsai.ugr.es>
 * @author Andrés Cano Utrera <acu@decsai.ugr.es>
 * @author Luis Castillo Vidal <L.Castillo@decsai.ugr.es>
 *
 * Created on 24 de octubre de 2025, 9:27
 */

#include <iostream>

#include <cstring>
#include <string>

#include "DataSet.h"
#include "Clustering.h"

using namespace std;

/**
 * Shows help about the use of this program in the given output stream
 * @param outputStream The output stream where the help will be shown (for example,
 * cout, cerr, etc)
 * @param message Additional message to show with the help
 */
void showHelp(std::ostream& outputStream, const string &message) {
    outputStream << "ERROR in Fraud4 parameters. " << message << endl;
    outputStream << "Run with the following arguments:" << endl;
    outputStream << "Fraud4 [-K1 <K1>] [-K2 <K2>] [-nr] " <<
        "[-o <classifiedDataset.dts>] <trainingDataset.dts> <testClassify.dts>" <<
        endl;
    outputStream << endl;
    outputStream << "Parameters:" << endl;
    outputStream << "-K1 <K1>: an integer with the number of clusters to " <<
        "use in the reduction dimensionality algorithm (5 by default)" << endl;
    outputStream << "-K2 <K2>: an integer with the number of neighbouring " <<
        "instances to use in the KNN algorithm (6 by default)" << endl;
    outputStream << "-nr: Indicates that dimensionality reduction " <<
        "should not be applied in the clustering algorithm " <<
        "(by default dimensionality reduction is done)" << endl;
    outputStream << "[-o <classifiedDataset.dts>]: name of the file where " <<
        "the classified dataset will be saved " <<
        "(tests/output/output.dts by default)" << endl;
    outputStream << "<trainingDataset.dts>: name of the dataset file used as " <<
        "model (training dataset)" << endl;
    outputStream << "<testClassify.dts>: name of the dataset file " <<
        "to be classified (test dataset)" << endl;
    outputStream << endl;
}

/**
 * The purpose of this program is to read two DataSet objects from dts files,
 * a training dataset and a test dataset, and to classify the instances in the
 * test dataset using the training dataset as model for the KNN algorithm.
 *
 * The number of clusters K1 to be used in the reduction dimensionality process
 * can be provided as an optional command line argument (if not provided,
 * K=5 is used).
 * The number of neighbours K2 to be used in the KNN algorithm can be provided
 * as an optional command line argument (if not provided, K=6 is used).
 * The name of the output dts file (classified dataset) can be provided to
 * the program as an optional command line argument (if not provided, the
 * output file is tests/output/output.dts).
 * Additionally, an optional argument can be provided to indicate that
 * dimensionality reduction should not be applied in the classification.
 * The name of the file with the training dataset is passed as the
 * penultimate command line argument.
 * The name of the file with the test dataset is passed as the last argument.
 * See below the Running sintax.
 *
 * The program begins by processing the command line arguments. Then, it loads
 * the training and test datasets from the given dts files. After that, it
 * classifies the instances in the test dataset using the KNN algorithm with the
 * provided K2 value and the training dataset as model. If indicated, before
 * classifying, the dimensionality of both datasets is reduced using clustering
 * with the provided K1 value. Finally, it saves the classified test dataset
 * in the given output dts file.
 *
 * Running sintax:
 * > build/Fraud4 [-K1 <K1>] [-K2 <K2>] [-nr] [-o <classifiedTestDataset.dts>]
 *                <trainingDataset.dts> <testDataset.dts>
 *
 * Running example:
 * > build/Fraud4 -K1 2 -K2 6 -o /tmp/dataset.dts ../Datasets/princeton_training.dts
 *                ../Datasets/princeton_test.dts
 */
int main(int argc, char* argv[]) {
    DataSet trainingDataset, // Dataset used as model to classify
        testDataset;     // Dataset to be classified

    // Name of the output file (tests/output/output.dts by default)
    string outputFileName="tests/output/output.dts";
```

```
    int K1 = 5, // Number of clusters for dimensionality reduction
        K2 = 6; // Number of nearest neighbours for KNN
    bool doReduction = true; // Indicates whether or not to do reduction dimensionality

    // Indicates if all the parameters starting with - has been read
    bool hasBeenReadInitialParameters = false;
    int indexInputFile = -1; // index of the input file in argv

    // Loop to process program arguments

    // Load training and test datasets

    // Classify the test dataset

    // Save the classified test dataset in the given output file

    return 0;
}
```